MONTPELLIER YNOV CAMPUS.

Mastère 2

DÉVELOPPER POUR LE CLOUD

Cours 1

Développer pour le Cloud

INTERVENANT
M. BENHAMDI Ayoub

DATE
20/03/2026

DURÉE
7 h (09h00–13h00 / 14h00–17h00)

PÉRIODE MODULE
20/03/2026 → 07/07/2026

OBJECTIF : Poser les fondations du cloud et du développement cloud-native

Objectifs & Programme

Cours 1 : Introduction et Fondamentaux

Objectifs Pédagogiques

- Comprendre les principes du cloud et architectures cloud-native
- Identifier les solutions adaptées aux besoins de développement
- Concevoir avec conteneurisation et architectures serverless
- Déployer et superviser sur AWS, GCP et Azure
- Automatiser avec des pipelines CI/CD
- Intégrer gestion de données et monitoring
- Documenter les déploiements et évolutions

Programme

7 HEURES

Introduction générale
Présentation, objectifs, évaluation

Fondamentaux Cloud
Définitions, IaaS/PaaS/SaaS, Modèles

Architectures Cloud-Native
Microservices, Conteneurs, Orchestration

Panorama Plateformes
Comparatif AWS, GCP, Azure

Travaux Pratiques
Setup environnement & comptes

Fondamentaux du Cloud Computing

Concepts Clés, Modèles de Service et Déploiement

Définition

Mise à disposition **à la demande** de ressources informatiques (calcul, stockage) via le réseau, avec une facturation à l'usage.

Caractéristiques Clés

- Élasticité & Scalabilité
- Self-service à la demande
- Paiement à l'usage
- Résilience & Haute dispo.
- Accès universel

Bénéfices

Time-to-market | Coûts optimisés | Focus métier

Modèles de Service

IaaS

INFRASTRUCTURE AS A SERVICE

Ressources brutes virtualisées : calcul (VM), stockage, réseau. Vous gérez l'OS et les applications.

AWS EC2, GCE

PaaS

PLATFORM AS A SERVICE

Environnement géré pour développer et déployer sans gérer l'infrastructure sous-jacente.

HEROKU, APP ENGINE

SaaS

SOFTWARE AS A SERVICE

Application complète fournie prête à l'emploi, accessible via navigateur.

GMAIL, OFFICE 365

Modèles de Déploiement

Public Cloud
Mutualisé, accessible à tous

Private Cloud
Dédié à une organisation

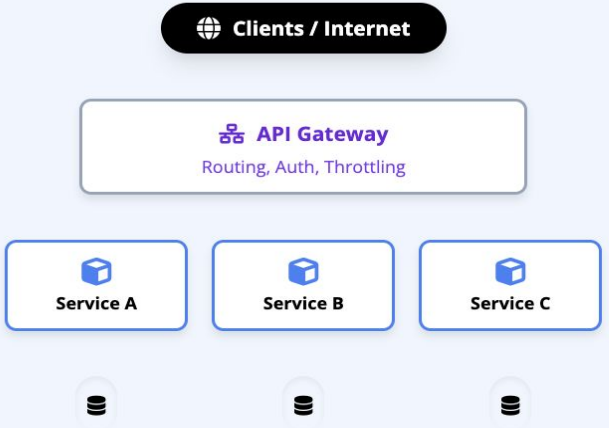
Hybrid Cloud
Mix Public + Privé

Multi-Cloud
Plusieurs fournisseurs publics

Architectures Cloud-Native

Microservices, Conteneurs et Orchestration

ARCHITECTURE MICROSERVICES



Principes Fondamentaux

- Immuabilité
- Scalabilité Horizontale
- Automatisation
- Tolérance aux pannes
- Observabilité

Monolithe vs Microservices

MONOLITHE

✗ Couplage fort, déploiement "tout ou rien", scalabilité complexe.

MICROSERVICES

✓ Découplage, déploiement indépendant, techno agnostique.

Conteneurisation & Orchestration

Les conteneurs (Docker) empaquettent code et dépendances pour la **portabilité**. L'orchestration (Kubernetes) gère le cycle de vie :

- Planification
- Service Discovery
- Auto-healing

Serverless (FaaS)

Exécution de fonctions déclenchées par des événements (HTTP, DB, File). **Zéro gestion d'infrastructure**, facturation à la milliseconde.

Panorama des plateformes Cloud

Comparatif des leaders du marché : AWS, GCP et Azure

AWS

CALCUL	EC2, ECS/EKS, Lambda
STOCKAGE	S3, EBS, EFS
DONNÉES	RDS, DynamoDB, Redshift
RÉSEAU	VPC, Route53, CloudFront
OPS	CloudWatch, X-Ray

GCP

CALCUL	Compute Engine, GKE, Cloud Functions
STOCKAGE	Cloud Storage, Persistent Disk
DONNÉES	Cloud SQL, Firestore, BigQuery
RÉSEAU	VPC, Cloud DNS, Cloud CDN
OPS	Cloud Logging, Monitoring

Azure

CALCUL	Virtual Machines, AKS, Azure Functions
STOCKAGE	Blob Storage, Disk Storage, Files
DONNÉES	Azure SQL, Cosmos DB, Synapse
RÉSEAU	VNet, DNS, CDN
OPS	Azure Monitor, App Insights

Critères de choix stratégiques

- Besoins Techniques
- Coûts & Tarification
- Conformité & Régions
- Écosystème & Outils
- Compétences Équipe

Travaux Pratiques & Prochaines Étapes

Mise en place de l'environnement Cloud et premiers pas

1 Préparation de l'environnement

- [ ] **Créer un compte "Free Tier"**
  Choisir au moins un fournisseur parmi les majeurs. Attention à la carte bancaire (ne pas dépasser les quotas gratuits).
  AWS Free Tier · GCP Free Tier · Azure Free
- [ ] **Installer la CLI & Outils locaux**
  Installation des outils indispensables pour interagir avec le cloud depuis le terminal.
  Docker Desktop · Git · AWS CLI / gcloud / az
- [ ] **Configuration Accès & Région**
  Créer un utilisateur IAM (pas de root !) et configurer la région par défaut (ex: `eu-west-3` pour Paris).

2 Exercices de prise en main

- [ ] **Explorer la Console Web**
  Naviguer dans le dashboard, localiser les services IAM, EC2/Compute et S3/Storage.
- [ ] **Configurer la CLI**
  Lancer `aws configure`, `gcloud init` ou `az login` et vérifier la connexion.
- [ ] **Cycle de vie d'une ressource**
  Créer un Bucket (S3/GCS) ou un Groupe de ressources via CLI, lister, puis **supprimer** pour éviter les frais.

3 À faire pour le cours 2

- Compte Cloud actif et validé
- CLI installée et configurée
- Capture d'écran du terminal :
  Commande réussie (ex: `aws s3 ls`)

Astuce : Configurez des alertes de facturation (Budgets) dès maintenant pour éviter les surprises.

Rappel Évaluation

Projet

AVEC SOUTENANCE

Le projet validera l'ensemble des compétences : conception, déploiement, automatisation et documentation.

Les 5 Caractéristiques du Cloud

Définition standard selon le NIST (National Institute of Standards and Technology)

NIST #1

Self-service à la demande

Provisionnement automatique des ressources (serveurs, stockage) sans nécessiter d'interaction humaine avec le fournisseur de services.

NIST #2

Large accès réseau

Les capacités sont disponibles sur le réseau et accessibles via des mécanismes standards (HTTP/HTTPS, API) par des clients variés (mobiles, PC, tablettes).

NIST #3

Mutualisation des ressources

Les ressources informatiques du fournisseur sont regroupées pour servir plusieurs consommateurs (modèle multi-tenant), avec une allocation dynamique selon la demande.

NIST #4

Élasticité rapide

Les capacités peuvent être provisionnées et libérées, parfois automatiquement, pour s'adapter rapidement à la demande. Pour le consommateur, les ressources semblent illimitées.

NIST #5

Service mesuré

L'utilisation des ressources est surveillée, contrôlée et facturée (pay-as-you-go). Cela fournit une transparence totale pour le fournisseur et le consommateur.

Avantages et Défis du Cloud

Comprendre les bénéfices et les points de vigilance

Avantages

BÉNÉFICES

Coûts Optimisés (CAPEX/OPEX)

Passage d'investissements lourds en capital (matériel) à des coûts opérationnels variables. Paiement uniquement à l'usage réel.

Scalabilité & Élasticité

Capacité à augmenter ou diminuer les ressources automatiquement selon la demande. Haute disponibilité globale native.

Innovation Accélérée

Accès immédiat à des services managés avancés (IA, Big Data, IoT) sans développement complexe.

Time-to-Market Réduit

Déploiement d'infrastructure en quelques minutes au lieu de plusieurs semaines. Focus sur le code métier.

Défis & Risques

VIGILANCE

Sécurité & Conformité

Gestion complexe de la responsabilité partagée. Respect des normes (RGPD, HDS) et localisation des données critique.

Vendor Lock-in

Dépendance forte aux technologies propriétaires d'un fournisseur, rendant la migration ou la portabilité difficile.

Gestion des Coûts (FinOps)

Risque de factures imprévues (frais de sortie, ressources oubliées). Nécessite une gouvernance financière stricte.

Complexité Réseau & Latence

Dépendance à la connectivité internet. Latence variable selon les régions. Architecture réseau distribuée plus complexe.

Introduction à Docker

Conteneurisation : Concepts, Cycle de vie et Commandes

CYCLE DE VIE DOCKER

1. Dockerfile
Recette de construction (Code)

↓

2. Image Docker
Artefact immuable (Build)

↓

3. Registry (Hub)
Stockage & Partage (Push/Pull)

↓

4. Conteneur
Instance en exécution (Run)

Concepts Clés

IMAGE
Modèle statique et immuable contenant l'OS, le code et les librairies.

CONTENEUR
Processus isolé et léger exécutant une instance d'une image.

Commandes Essentielles

Construire une image

```
$ docker build -t mon-app:v1 .
```

Lancer un conteneur (détaché + ports)

```
$ docker run -d -p 8080:80 mon-app:v1
```

Gérer le cycle de vie

```
$ docker ps / logs / stop / rm
```

Pourquoi utiliser Docker ?

- **Portabilité :** "Build once, run anywhere" (Dev, Test, Prod)
- **Isolation :** Gestion propre des dépendances (pas de conflits)
- **Reproductibilité :** Environnements identiques pour toute l'équipe
- **CI/CD & K8s :** Brique de base pour l'automatisation et l'orchestration

Services Cloud par Catégorie

Panorama des briques fondamentales d'une architecture Cloud

Compute (Calcul)

- **Machines Virtuelles (VMs) :** IaaS standard (EC2, GCE, Azure VM).
- **Conteneurs :** Orchestration gérée (EKS, GKE, AKS, ECS).
- **Serverless :** FaaS, exécution événementielle (Lambda, Functions).
- **Batch :** Traitements par lots planifiés.

Stockage

- **Objet :** Données non structurées, scalable à l'infini (S3, Blob, GCS).
- **Bloc :** Volumes disques attachés aux VMs (EBS, Persistent Disk).
- **Fichier :** Systèmes de fichiers partagés (EFS, Azure Files).
- **Archivage :** Stockage froid à bas coût (Glacier).

Bases de Données

- **SQL Managé :** Relationnel sans gestion d'OS (RDS, Cloud SQL).
- **NoSQL :** Clé-valeur, document, scalable (DynamoDB, CosmosDB).
- **Cache :** In-memory pour la performance (ElastiCache, Redis).
- **Data Warehouse :** Analytique (Redshift, BigQuery).

Réseau

- **VPC / VNet :** Réseau virtuel isolé et sous-réseaux.
- **Load Balancing :** Répartition de charge (ALB, GLB).
- **CDN :** Distribution de contenu (CloudFront).
- **DNS :** Gestion de nom de domaine (Route53).

Observabilité

- **Monitoring :** Métriques et tableaux de bord (CloudWatch).
- **Logging :** Centralisation des logs applicatifs et système.
- **Tracing :** Suivi des requêtes distribuées (X-Ray).
- **Alerting :** Notifications sur seuils critiques.

Architecture Modulaire

Une architecture cloud robuste combine ces services comme des briques LEGO pour répondre aux besoins spécifiques de performance, coût et sécurité.

Introduction au Serverless

Architecture Event-Driven et FaaS

FLUX EVENT-DRIVEN

Événement (Trigger)
HTTP, Upload fichier, Timer, DB change

Fonction (FaaS)
Logique métier éphémère

Node.js Python Java

Action / Réponse
Stockage DB, Notification, Retour API

Définition & Principes

Modèle d'exécution où le cloud provider gère l'allocation dynamique des ressources. Le code s'exécute dans des conteneurs éphémères déclenchés par événements.

Facturation à l'exécution | Scalabilité Auto | Stateless | NoOps

Cas d'Usage

- APIs REST/GraphQL (Backend for Frontend)
- Traitement de fichiers (Image resizing)
- ETL & Batch Processing
- Automatisation IT (Cron jobs)

Avantages Clés

- **Coûts** : Pay-per-use (idle = 0€)
- **Vitesse** : Focus code vs infra
- **Scale** : De 0 à 10k requêtes instantanément

Limitations à considérer

Cold Starts : Latence au premier appel.
Contraintes : Timeouts (ex: 15min max), Mémoire limitée.
Complexité : Debugging distribué et monitoring plus ardu.

PRINCIPALES SOLUTIONS

AWS Lambda | Azure Functions | Cloud Functions

Sécurité et Bonnes Pratiques Cloud

Protection, Conformité et Résilience des Architectures

Modèle de Responsabilité Partagée

Fournisseur Sécurité *du* cloud (infrastructure physique, hyperviseur, réseau global).

- **Client** Sécurité *dans le* cloud (données, configurations, accès, OS invité).

Identités & Accès (IAM)

- Principe du **moindre privilège** **(donner uniquement les droits nécessaires).**
- Utilisation de **MFA** (Multi-Factor Authentication) pour tous les utilisateurs.
- Privilégier les rôles temporaires aux clés d'accès statiques.

Sécurité Réseau

- Segmentation stricte (VPC, Subnets publics/privés).
- Security Groups / NACLs comme pare-feu virtuels.
- Approche **Zero Trust** et utilisation d'endpoints privés.

Protection des Données

- Chiffrement **au repos** (disques, S3) et **en transit** (TLS/SSL).
- Gestion centralisée des clés via KMS (Key Management Service).
- Gestion des secrets applicatifs (pas de mots de passe en dur).

Observabilité Sécurité

- Centralisation des logs (CloudTrail, VPC Flow Logs).
- Détection d'anomalies et d'intrusions (GuardDuty, Security Center).
- Alertes automatisées et plans de réponse aux incidents.

Conformité & Résilience

- Sauvegardes régulières et tests de restauration (DRP).
- Durcissement des configurations (CIS Benchmarks).
- Audit continu de la conformité (AWS Config, Azure Policy).